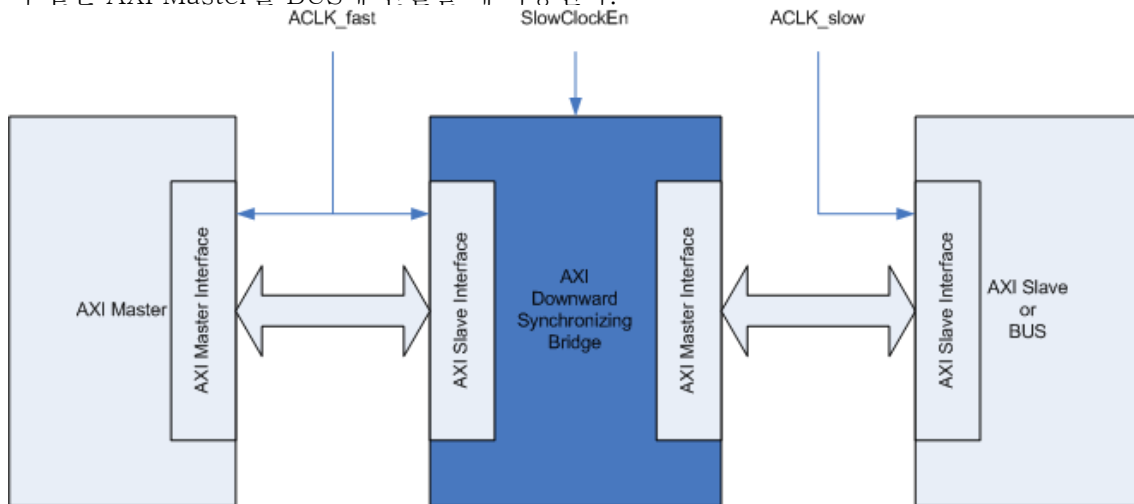


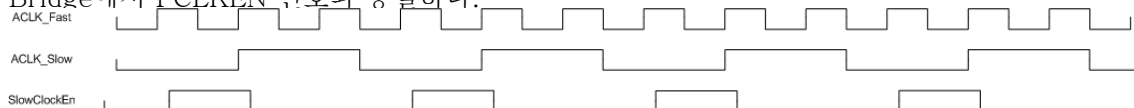
Title	AXI downward synchronizing Bridge
Project	None
Description	AXI downward synchronizing Bridge의 설계와 동작
Author	holelee
Date	19/May/2005

(*) AXI Downward Synchronizing Bridge란?

AXI Downward Synchronizing Bridge(이하 DS Bridge)는 clock이 빠른 AXI master를 clock이 느린 AXI slave에 연결해 주기 위한 장치이다.(아래 그림 참조) AXI Master가 AXI Slave에 비해 clock 주파수 빠른 경우에는 항상 사용할 수 있지만, 주로 빠른 clock으로 동작하는 CPU와 같은 AXI Master를 BUS에 연결할 때 사용된다.



기본적으로 빠른 clock과 느린 clock 사이에는 정수배의 주파수 차이가 있어야 하고 느린 clock의 rising edge는 빠른 clock의 rising edge의 시점과 일치해야 한다. DS Bridge 자체는 느린 clock을 입력으로 받지 않는 대신 느린 clock의 rising edge의 시점을 알 수 있도록 SlowClockEn 이라는 신호를 받는다. SlowClockEn이 '1'이면 다음에 오는 빠른 clock의 rising edge에서 느린 clock이 역시 rise한다는 의미를 지닌다.(아래 그림 참조) 이는 AXI2APB Bridge에서 PCLKEN 신호와 동일하다.



(*) AXI DS Bridge의 동작 목표

DS Bridge가 주로 CPU와 같은 빠른 AXI Master를 비교적 느린 BUS에 연결하기 위해서 사용된다는 가정하에서는 DS Bridge 내부에 FIFO를 사용하는 data buffering은 필요없게 된다. 우

선 Read의 경우에는 buffering을 한다고 해서 응답성이 좋아지거나 bandwidth를 더 확보할 수 없다. Write의 경우에도 AXI Master가 BUS가 아닌 이상 write data interleaving을 한다거나 할 필요가 없다. 따라서 FIFO등의 장치를 사용하지 않는 단순한 장치로 개발을 진행하기로 한다.

ARM9과 같은 CPU는 AHB2AXI Bridge를 사용하게 되는데 AHB2AXI Bridge가 combinational한 path를 가지게 되므로 DS Bridge에서 timing isolation을 수행할 수 있도록 한다. Slow AXI interface와 Fast AXI interface 사이에는 모든 signal이 register로 latching 되도록 구성한다.

(*) 설계 방법

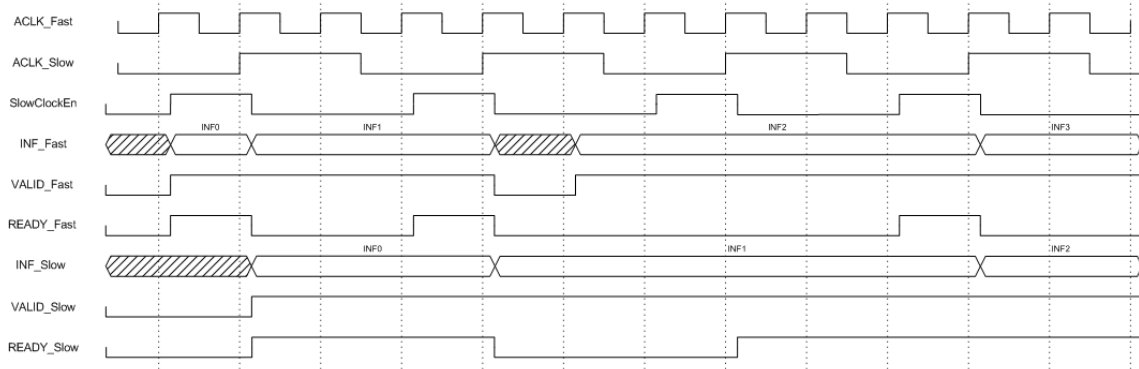
AXI는 총 5개의 채널을 가지고 있고 그 중 3개는 Master에서 Slave쪽으로 정보가 흐르고, 나머지 2개는 반대로 Slave쪽에서 Master쪽으로 정보가 흐르게 된다. 따라서 모든 channel을 따로 코딩할 필요는 없고 두가지 경우 만을 구분하여 코딩하면 될 것으로 보인다. AXI Register Slice의 설계와 마찬가지로 VALID,READY,INFORMATION(가변bit width)을 이용하여 두 가지 경우에 대해서 coding한 후에 모두 5개의 채널을 사용할 수 있도록 instantiation만 하면 된다.

Test는 Random vector를 생성하여 채널을 통해서 전달되는 값을 비교하는 식으로 진행한다.

(*) Master(Fast side)에서 Slave(Slow side)로 정보가 가는 경우의 timing

- Simple(Registered Forward : 정보만 latch되고 READY는 아님)

Master에서 Slave쪽으로 정보가 흐르는 경우의 timing은 다음과 같다.



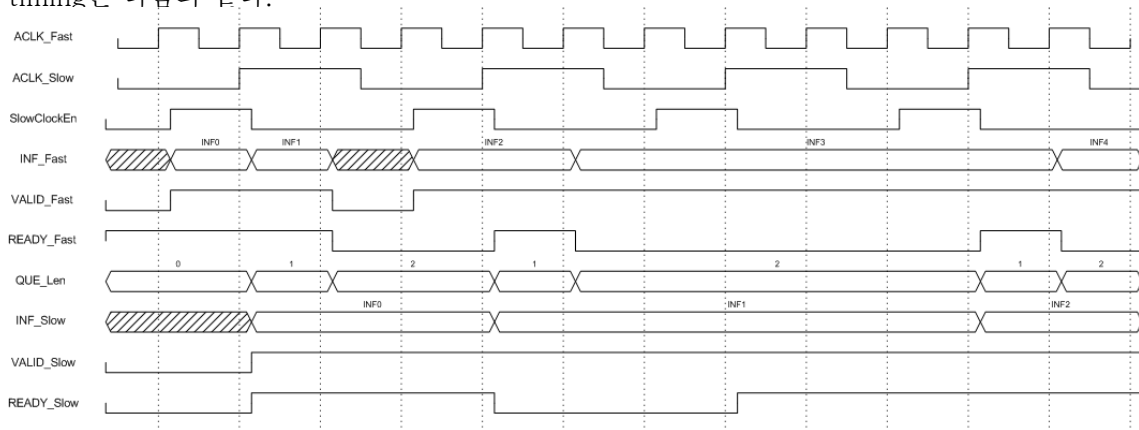
동작은 비교적 간단하다. INF_Slow와 VALID_Slow는 특정 조건에서 INF_Fast와 VALID_Fast를 latch한 후에 출력한다. READY_Fast는 SlowClockEn이 high가 되었을 때만 high가 된다. 물론 Slave가 Valid한 information을 받지 못하고 READY_Slow가 low로 유지되는 기간에는 READY_Fast도 low로 출력되지만 그 외에는 SlowClockEn이 high이면 READY_Fast는 high가 된다.

여기에는 한 가지 문제가 있는데 READY_Slow 입력이 SlowClockEn과 같이 combinational하

게 READY_Fast로 출력된다는 점이다. Information(INF_Fast, INF_Slow)은 register로 latch 되지만 READY_Slow는 그렇지 않다. AXI Register Slice에서 registered forward와 같은 형태로 보면 된다. SlowClockEn이 항상 '1'이라면 AXI Register Slice인 registered forward와 완전히 동일하다.

- Advanced(Fully Registered)

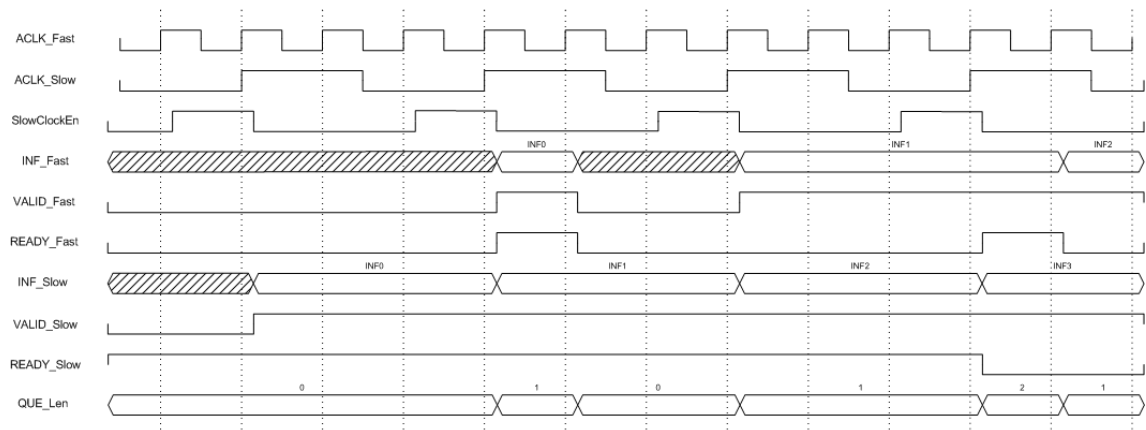
Simple 버전에서 READY 신호가 timing isolation이 이루어지지 않은 문제를 해결하기 위해서는 Information을 담을 수 있는 depth 2짜리 QUE를 두고 그것을 이용하면 된다. 이런 경우의 timing은 다음과 같다.



기본적으로 QUE가 full이 아니면 READY_Fast를 high로 하여 Master(Fast side)의 Information을 계속 받아 둔다. Slave(Slow side)가 VALID한 정보를 accept하게 되면 Que의 length가 감소하게 된다. READY_Fast가 Que의 length에만 연관을 가지고 timing isolation이 이루어지게 된다. Simple 방법에 비해 두 배 정도의 gate count를 가질 것으로 예상되지만 완전한 timing isolation을 이루게 된다는 점에서 선호된다. 또한 Master쪽의 응답은 빨라지므로 Read Address Channel과 Write Address channel에서의 사용에 있어서는 좀더 좋을 수 있다. SlowClockEn이 항상 high가 되면 Register Slice중에 Fully registered mode와 동일하게 동작하게 된다.

(*) Slave(Slow side)에서 Master(Fast side)로 정보가 가는 경우의 timing

Slave에서 Master로 정보가 가는 경우의 timing은 다음과 같다.



Slave쪽으로 출력하는 READY_Slow는 Slow Clock에 동기되도록 생성하게 된다. 위의 timing 마지막 부분에서 INF1을 Master가 READY_Fast를 high로 해서 바꾼다고 해서 바로 READY_Slow가 high가 되지 않도록 한다. 이런 구성을 가지면서 Slow side쪽의 bandwidth를 완전하게 이용하기 위해서는 Information을 저장할 수 있는 Depth 2짜리 Que가 있어야 하고 그 Length가 위의 timing상 QUE_Len이라는 signal이다. AXI Register Slice에서 Fully registered와 비슷한 구성이고, Master쪽의 READY가 combinational하게 Slave쪽의 READY로 출력되지 않는다. 따라서 모든 signal에 대해서 timing isolation이 이루어지게 된다.